

재생에너지 기반 AI 스마트팜 제어 시스템

AI 예측 기반 에너지 최적화 스마트팜

팀장	김정현
팀원	백동윤
팀원	이수빈



목차



01	프로젝트 개요 및 진행 일정	03p
02	하드웨어 설계	06p
03	AI 모델 설계	17p
04	웹 대시보드	27p

01

프로젝트 개요 및 진행 일정

- 프로젝트 개요
- 프로젝트 진행 일정



프로젝트 개요



프로젝트 배경

배경: 태양광은 기상 영향으로 발전량 변동이 커 스마트팜 전력 안정성 확보가 어려움

본 프로젝트: IoT+GRU로 발전량을 예측해 에너지 모드를 자동 최적화함

기존 스마트팜과 차이점: 기존 IoT는 반응형 제어, 본 시스템은 예측 기반 선제 제어

프로젝트 목표

- ✓ 소형 태양광 하이브리드 시스템 구축
- ✓ 실시간 데이터 수집 및 모니터링 구현
- ✓ PyTorch 기반 시계열 모델로 발전량 예측
- ✓ 웹 대시보드를 통한 실시간 시각화 및 제어 구현

팀 구성

이름	담당	담당 업무
김정현(팀장)	웹	서버 구축, DB 관리, 실시간 모니터링 대시보드 개발
백동운	하드웨어	발전 시스템 구축, 센서 연결, 데이터 수집
이수빈	AI	데이터 전처리, PyTorch 모델 설계/학습, 성능 평가

프로젝트 진행 일정



하드웨어 구성	1	프로젝트 계획서 작성, 부품 구매, 회로 설계 초안		
하드웨어 설계	2	회로 설계, 센서 연결, 배터리 발전 시스템 구축		
데이터 수집 및 전처리 / AI 학습	3	회로 설계	전처리 파이프라인, GRU, XGBoost 학습	
	4	개발 환경 구성, 실시간 대시보드, AI 추론 연동		
성능 평가 / 발표 준비	5	MAE 평가, PPT 작성		

02

하드웨어 설계

- HW 설계 목표
- 전체 회로도
- 이슈



HW 설계 목표



외부 전원 ZERO

외부의 전원없이 태양광으로부터 발전한 전기를 ESS에 저장하고, 스마트팜 운영에 사용할 전력을 ESS에서 사용

지능형 에너지 관리 전략 (EMS) 최적화

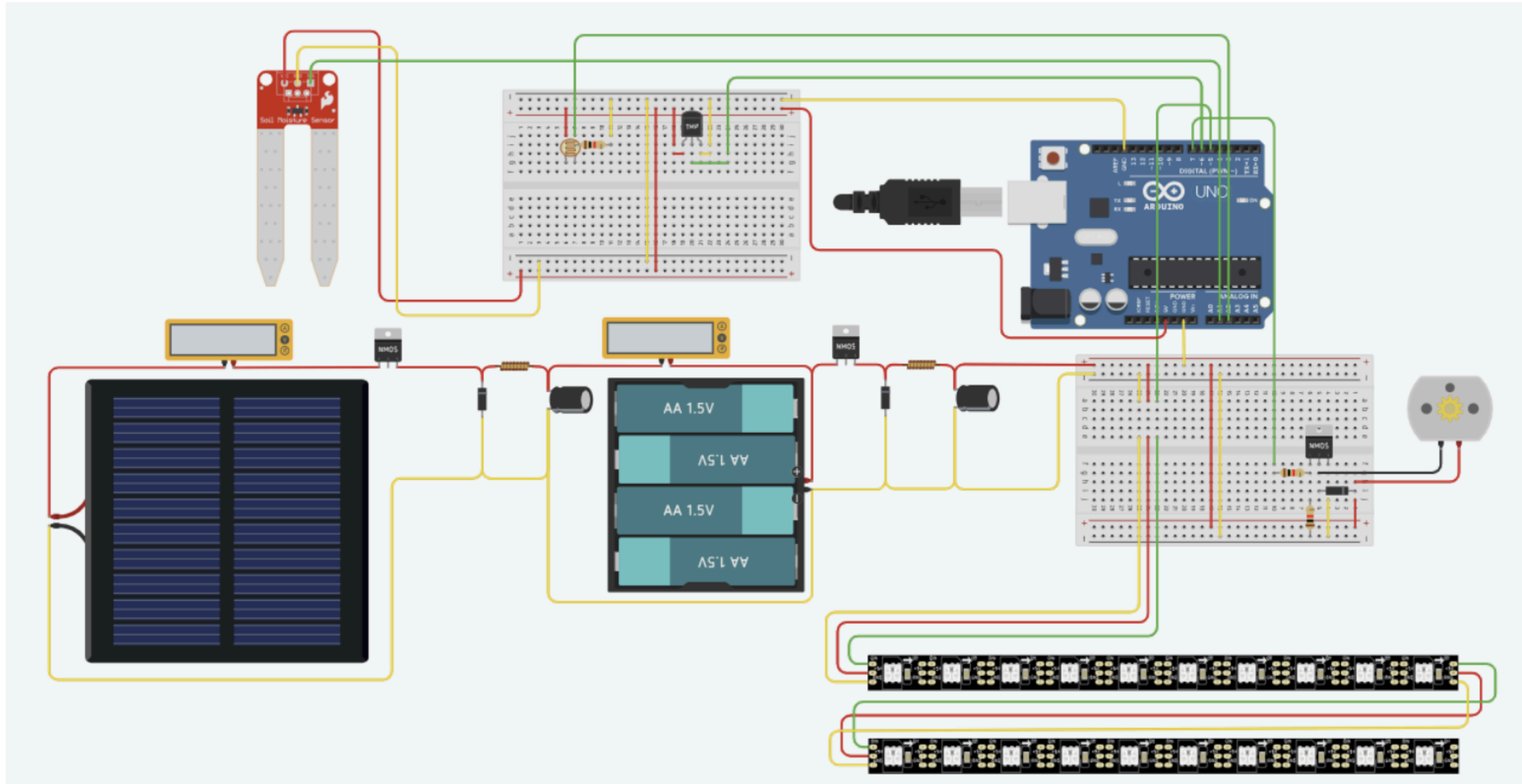
배터리 SOC(State of Charge)를 전압값으로 환산하여 에너지 잔량에 따라 시스템 운영 모드(Emergency/Eco/Standard)를 자동 전환함으로써, 제한된 에너지 내에서 시스템의 가동 효율과 지속 가능성을 극대화함.

HW 부품 리스트

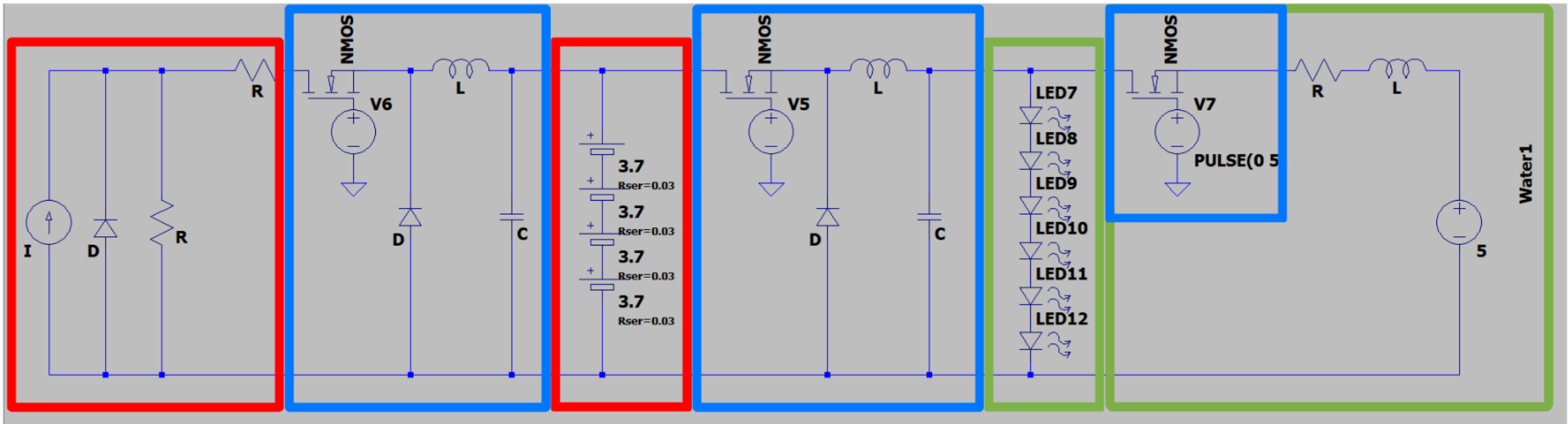


소자 종류	개수	소자종류	개수
태양광 패널 110X60	8	5V 급수모터	1
18650배터리 (메가캐논, Samsung SDI)	4	3채널 전압-전류 센서 INA 3221	1
DC-DC 컨버터 XL 4015(CC-CV) / XL 4015(CV)	2	토양습도 센서	1
4채널 5V 릴레이 모듈 SZH-RLBG-012	1	온-습도 센서 DHT 22	1
Neo Pixel strip	1m (led: 60개)	조도 센서	1

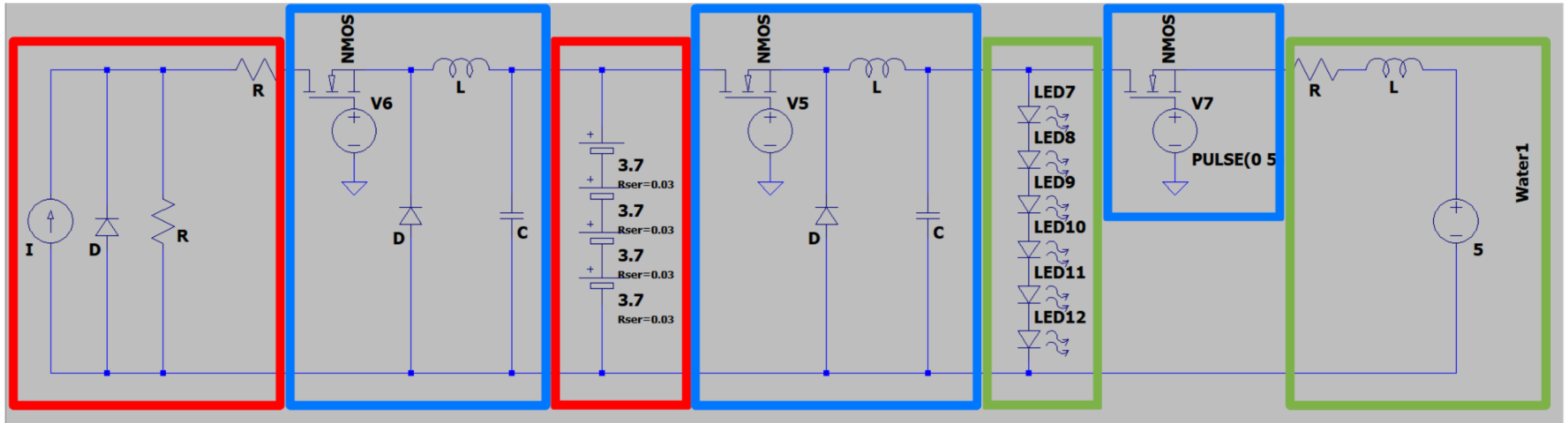
전체 회로도



전체 회로도

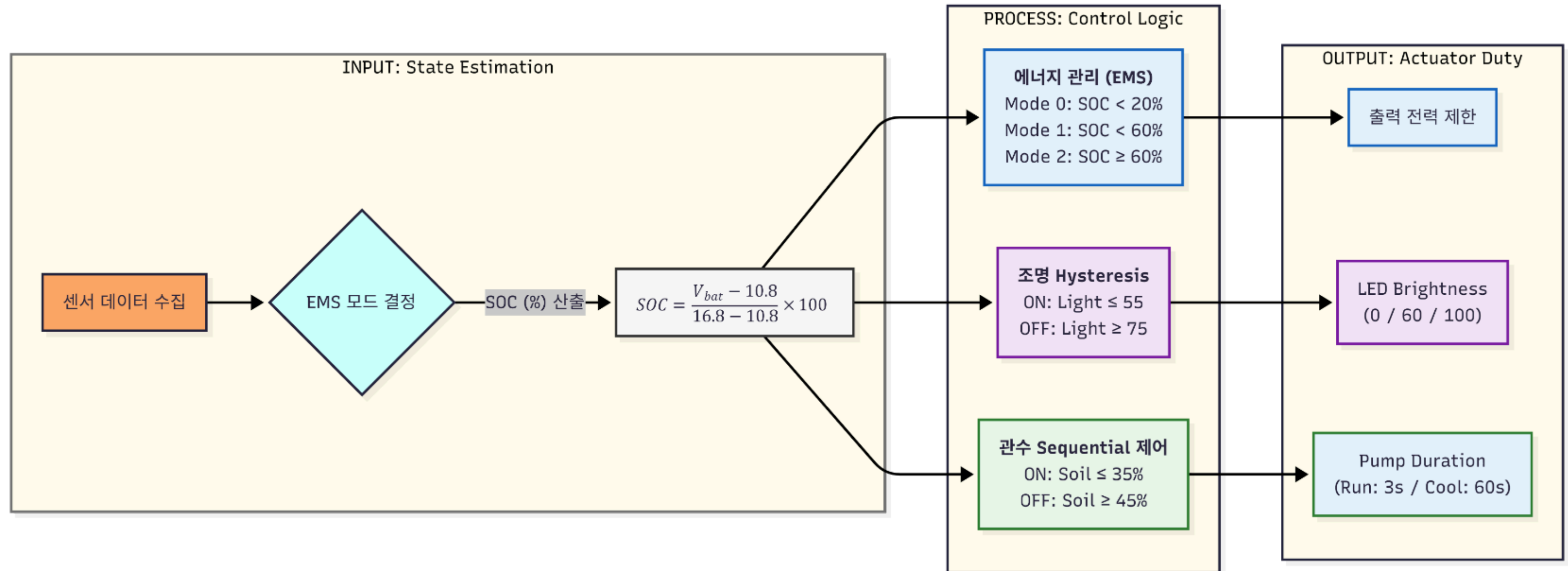


태양광(발전부) -> DC-DC 컨버터 -> ESS(저장부)
-> DC-DC 컨버터 -> LED & 급수모터(출력부)



태양광 패널(4S-2P)	DC-DC Buck 컨버터	18650 배터리(4S)	DC-DC Buck 컨버터	Neo Pixel LED	N-MOSFET	급수 모터
110X60: 8개 최대전압: 6(V/개당) -> 24(V) 개방전압: 약 7(V/개당) -> 28(V) 최대전류: 0.17(A) -> 0.34(A)	XL 4015(CC-CV) 태양광으로부터 발전한 전기를 배터리에 충전 충전조건 충전전압(CV): 16.8(V) 충전전류(CC): 0.1(A)	18650: 4개 정격전압: 3.7(V/개당) -> 14.8(V) 충전전압: 4.2(V/개당) -> 16.8(V) 충전전류: 1~2(A)	XL 4015(CV) 배터리로부터 부하에 전력 공급 부하 전압: 5V	1M/LED 60개 식물의 성장 및 광합성에 필요한 파장 출력	IRFZ44NPBF 급수모터의 출력제어	식물에 수분 공급

제어 알고리즘



이슈



풍력 출력 문제

간이 풍력 발전기의 블레이드는 EVA폼으로 매우 강한 바람이 아닌 이상, 일반 바람에 의한 발전이 거의 불가함. -> 풍력 발전기 제외

WIFI 모듈

WIFI 모듈 연결 시 모듈의 발열과 센서들의 값들이 튀는 현상 발견
-> WIFI 모듈 제외

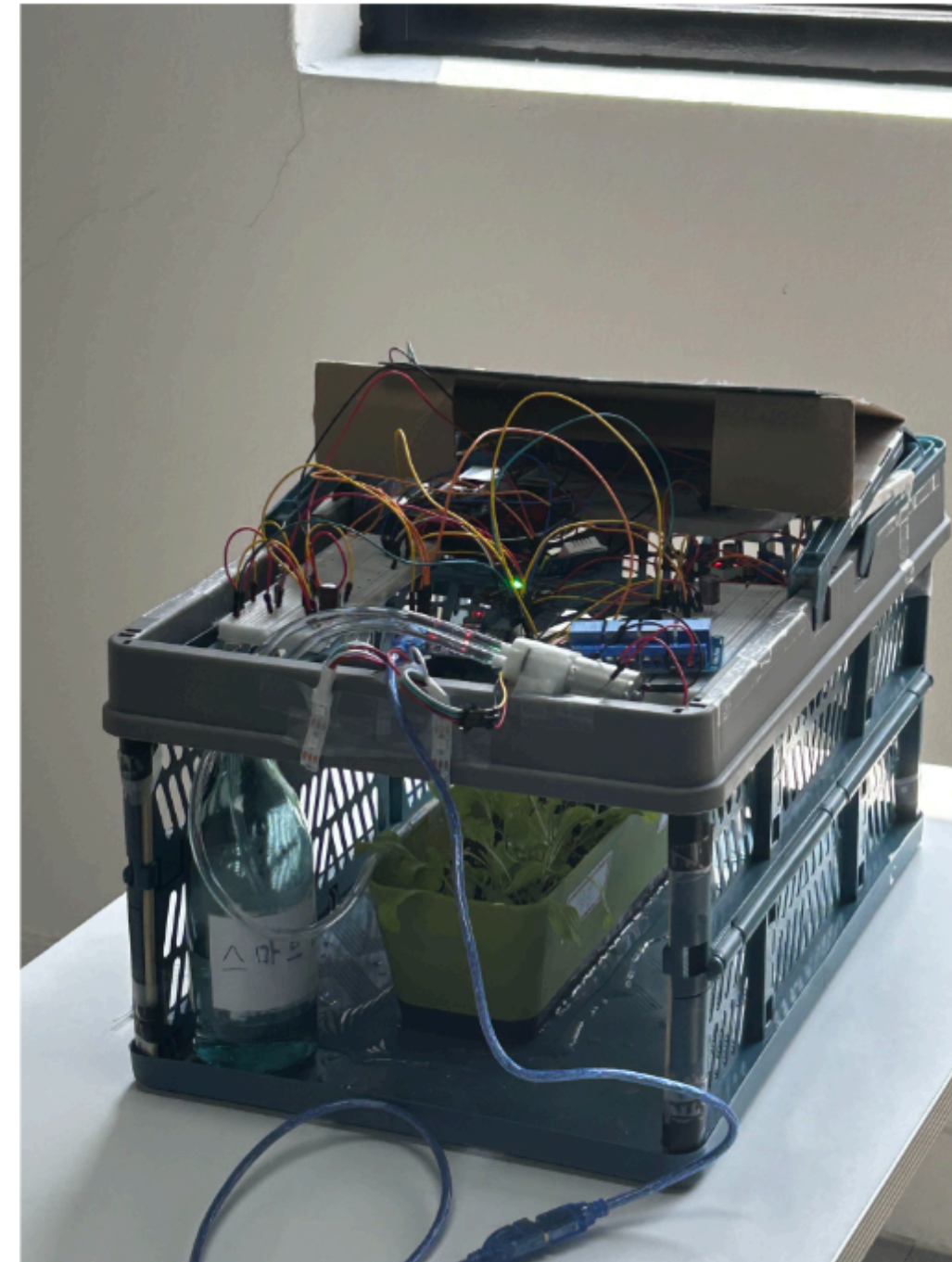
태양광 출력으로 인한 ESS 충전 불가

태양광에서 발전되는 전류는 평균 약 0.07(A)로, 이 것 또한 불안정한 출력으로 인해 정전류(CC)를 충족하지 못했기 때문에 배터리를 충전하지 못함.

스마트팜 외부



(좌) 스마트팜 뒷면



(우) 스마트팜 앞면

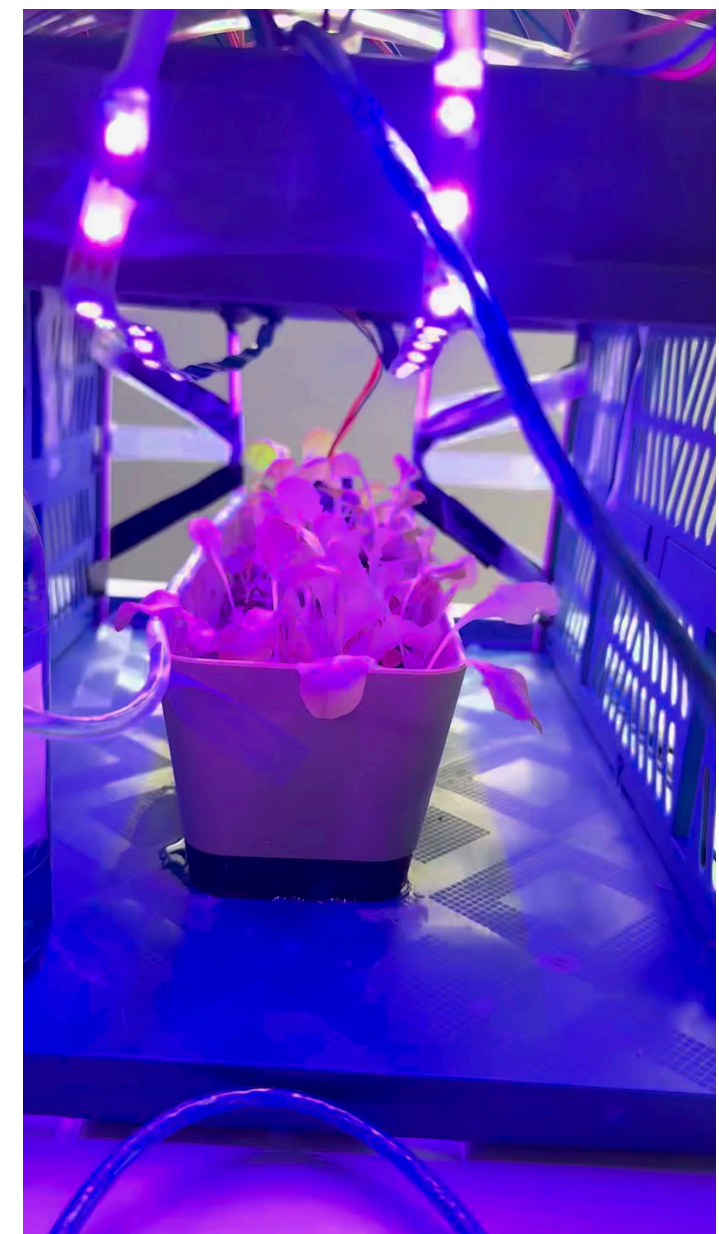
스마트팜 동작(밤 & 토양습도 40%이하)



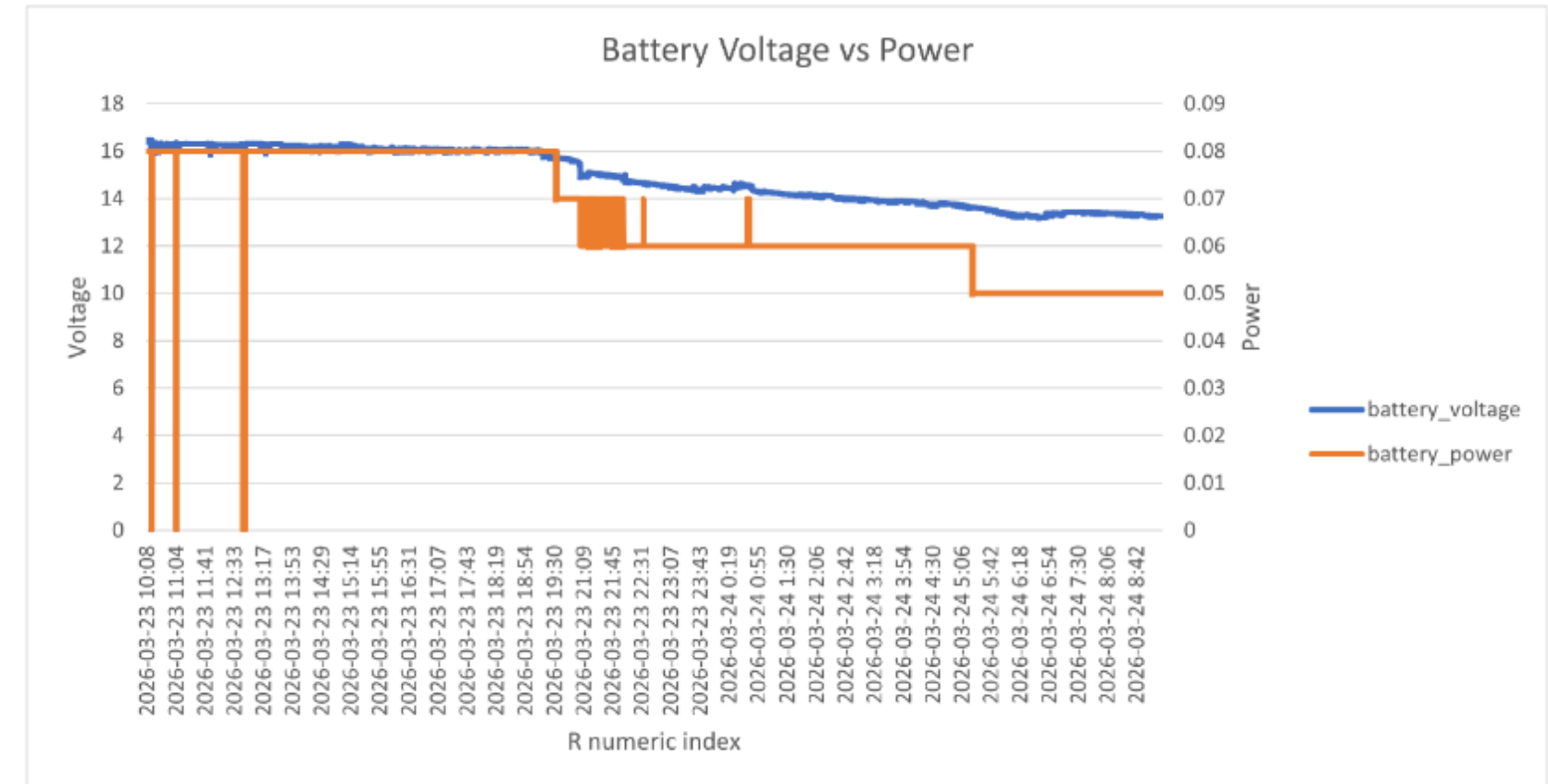
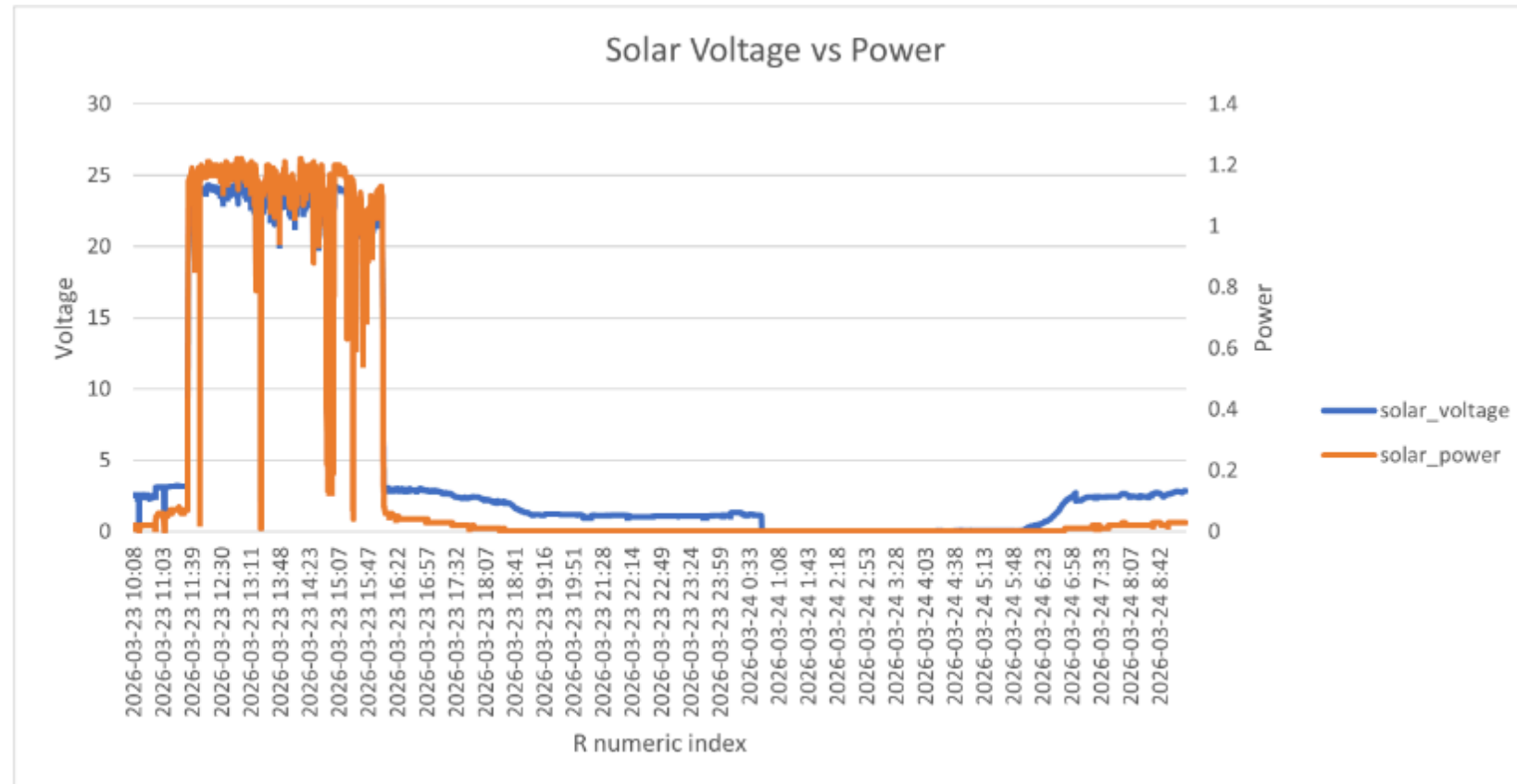
(좌) 스마트팜 LED



(우) 스마트팜 LED & 급수모터



태양광 및 ESS 전압, 전력



- 태양광의 전압은 최대 28V로 정전압(CV) 조건이 되지만,
- 전류는 최대 0.09(A) 평균 0.06~7(A)이며 전류 출력이 불안정해 정전류(CC) 조건을 충족하지 못했음.
- 배터리의 충전전압인 16.8(V)을 맞췄으나 목표 충전전류인 0.1(A)가 못되어 배터리 충전이 어려움

03

AI 모델 설계 및 학습 결과

-
- 데이터 수집 및 전처리
 - GRU 및 XGBoost 모델 설계와 학습 결과
 - GRU vs XGBoost 모델 비교



데이터 수집



수집 방식

아두이노가 10초마다 센서값을 JSON 형식으로 시리얼 출력하면, PC에서 실행중인 Flask 서버(app.py)가 이를 읽어 seonsor_log.csv와 sensor_log.json에 자동저장함

수집 항목

Timestamp / temperature / humidity / soil / light / solar_voltage / solar_current / solar_power / battery_voltage / battery_current / battery_power / soc / mode / water_alert

수집 결과

원본 데이터: 63,937행 / 10초 간격
5분 다운샘플링 후: 2,178행, 전처리 후 최종: 2,143행 (이상값 제거 후)

데이터 전처리



1) 5분 다운 샘플링 - 원본 10초 데이터를 5분 평균으로 압축

```
n = len(df)
train_df = df.iloc[:int(n * 0.8)]
test_df = df.iloc[int(n * 0.8):]
```

10초 데이터는 노이즈가 많고 1시간 후 예측에는 순간값보다 추세가 중요
5분 × 12 = 60분이므로 `shif(-12)`로 1시간 후 발전량을 타겟으로 생성

2) 시간 sin / cos 인코딩

```
df["hour_sin"] = np.sin(2 * np.pi * df["hour"] / 24)
df["hour_cos"] = np.cos(2 * np.pi * df["hour"] / 24)
```

시간(0~23)을 그대로 쓰면 23시 ↔ 0시 거리가 23으로 인식됨
sin/cos로 변환하면 연속적으로 표현되며 각 시각이 고유한 좌표를 가져 모델이 올바르게 학습

데이터 전처리



3) MinMaxScaler 정규화 (GRU만)

변수마다 단위가 달라 GRU 학습 불안정 → 모두 0~1로 통일

Fit은 train만, test는 transform만 적용해 데이터 누출 방지

```
f_scaler = MinMaxScaler()
train_X = f_scaler.fit_transform(train_df[FEATURES]) # 기준 계산 + 변환
test_X   = f_scaler.transform(test_df[FEATURES])    # 변환만
```

XGBoost는 값의 대소 관계만 보기 때문에 정규화 불필요

데이터 전처리



4) Lag/Rolling Feature (XGBoost만)

GRU는 시퀀스 구조로 과거를 내부적으로 기억

XGBoost는 현재 시점 한 줄만 받아서 과거 패턴을 직접 컬럼으로 만들어줘야 함

```
df["power_lag_1"] = df["solar_power"].shift(1)    # 5분 전
df["power_lag_2"] = df["solar_power"].shift(2)    # 10분 전
df["recent_mean_power_3"] = df["solar_power"].rolling(3).mean() # 최근 15분 평균
df["power_diff_1"] = df["solar_power"].diff(1)    # 5분간 변화량
```

이걸 안 하면 XGBoost는 과거를 전혀 모르고 현재 시점만 보고 예측하게 됨

예) "5분 전보다 발전량이 올라가고 있다" 같은 추세 정보를 모델이 숫자로 받을 수 있음

GRU 모델 설계



LSTM → GRU 변경 이유:

GRU는 LSTM보다 구조가 단순해 파라미터 수가 적고 학습이 빠름

소규모 데이터에서 과적합 위험이 낮고 성능은 비슷함

입력 변수 9개:

solar_power / solar_voltage /
solar_current / light / temperature /
humidity / hour_sin / hour_cos / soc

모델 구조:

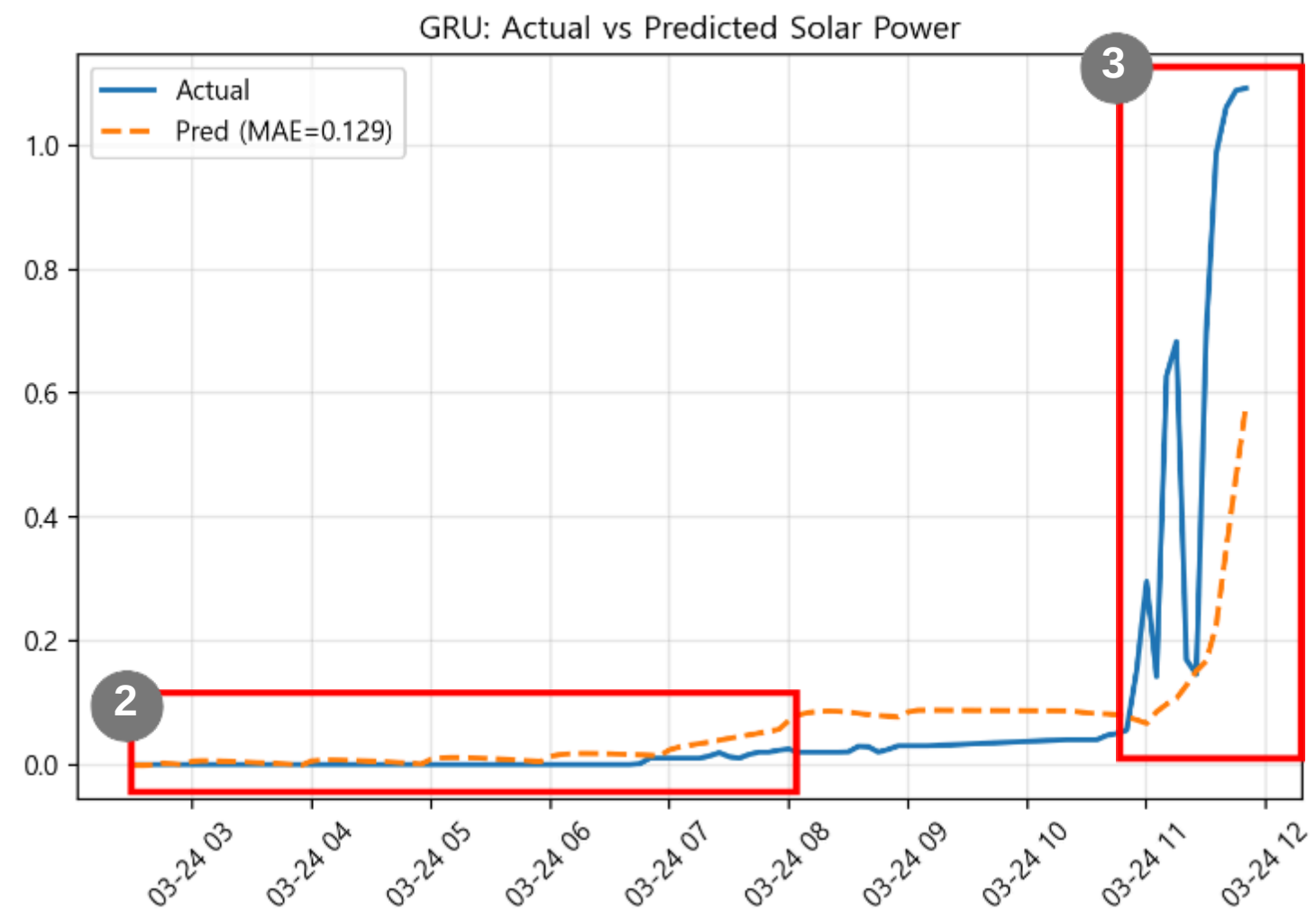
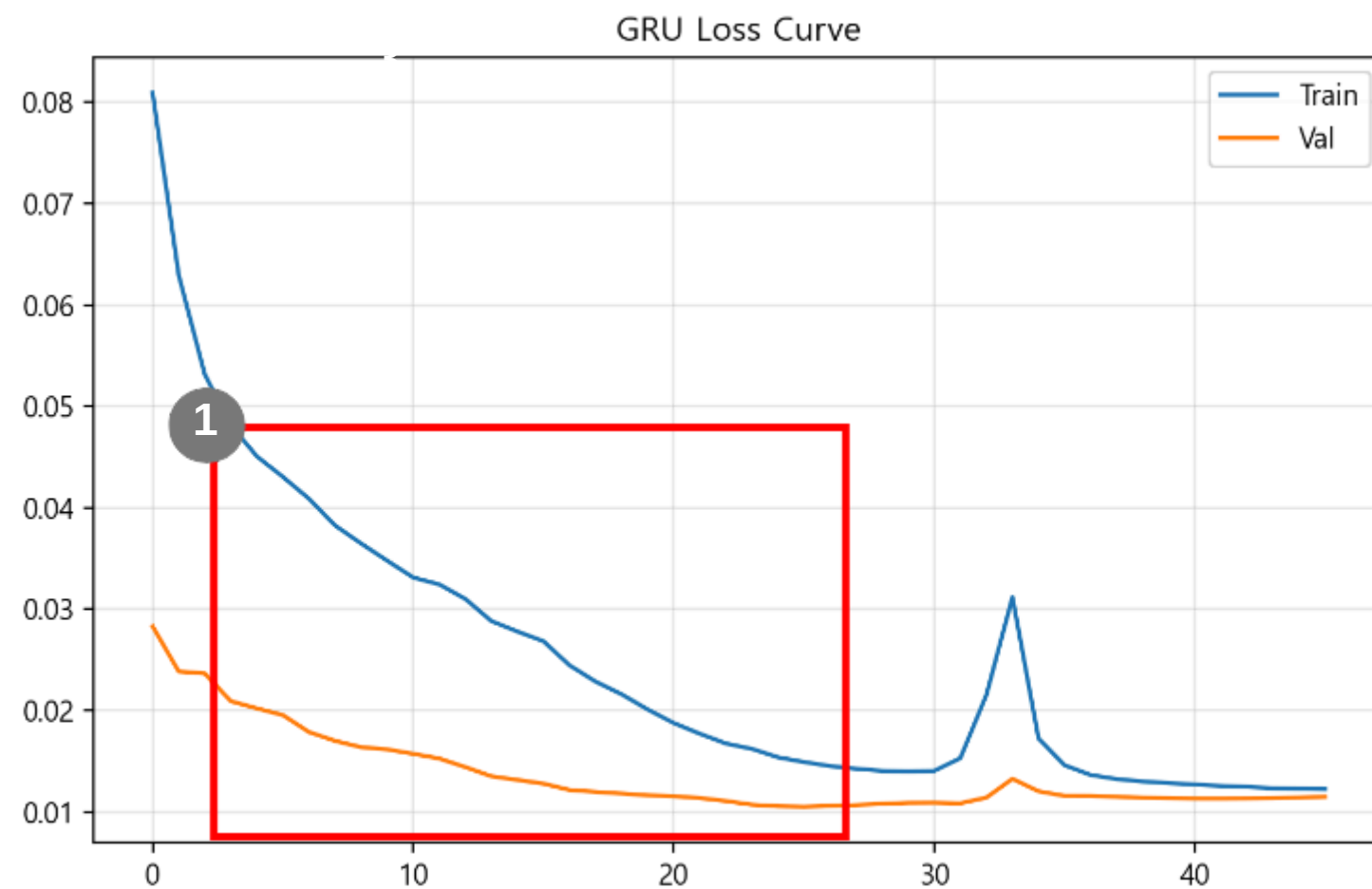
입력(9) → GRU(hidden=32) →
FC(32 → 16) → ReLU → FC(16 → 1) →
예측값(W)

학습 설정: MSELoss / Adam(lr=0.001) / Early Stopping(patience=20)

GRU 결과



GRU Loss Curve + 실제 vs 예측 그래프



- 1 Loss Curve: Train/Val 함께 수렴 → 과적합 없이 안정적으로 학습
- 2 야간 구간: 실제·예측 모두 0 → 정확히 학습
- 3 발전 피크(11~12시): 실제가 급격히 상승하지만 예측이 따라가지 못함 → 급변 패턴 학습 부족 한계

XGBoost 모델 설계



왜 XGBoost에만 피처 엔지니어링?

GRU는 시퀀스 구조로 과거를 내부적으로 기억

XGBoost는 현재 시점 한 줄만 입력받으므로 과거 패턴을 직접 컬럼으로 생성해야 함

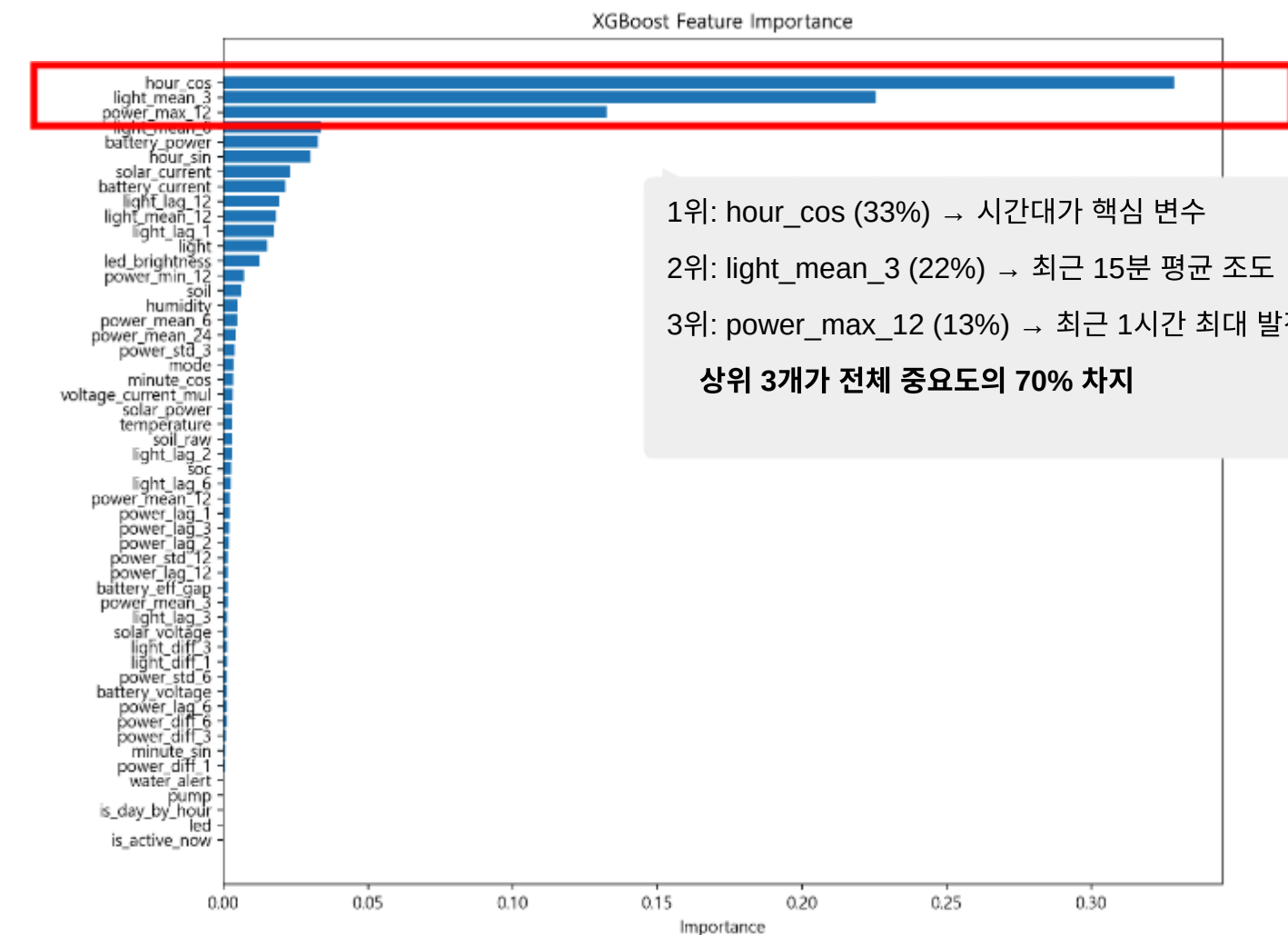
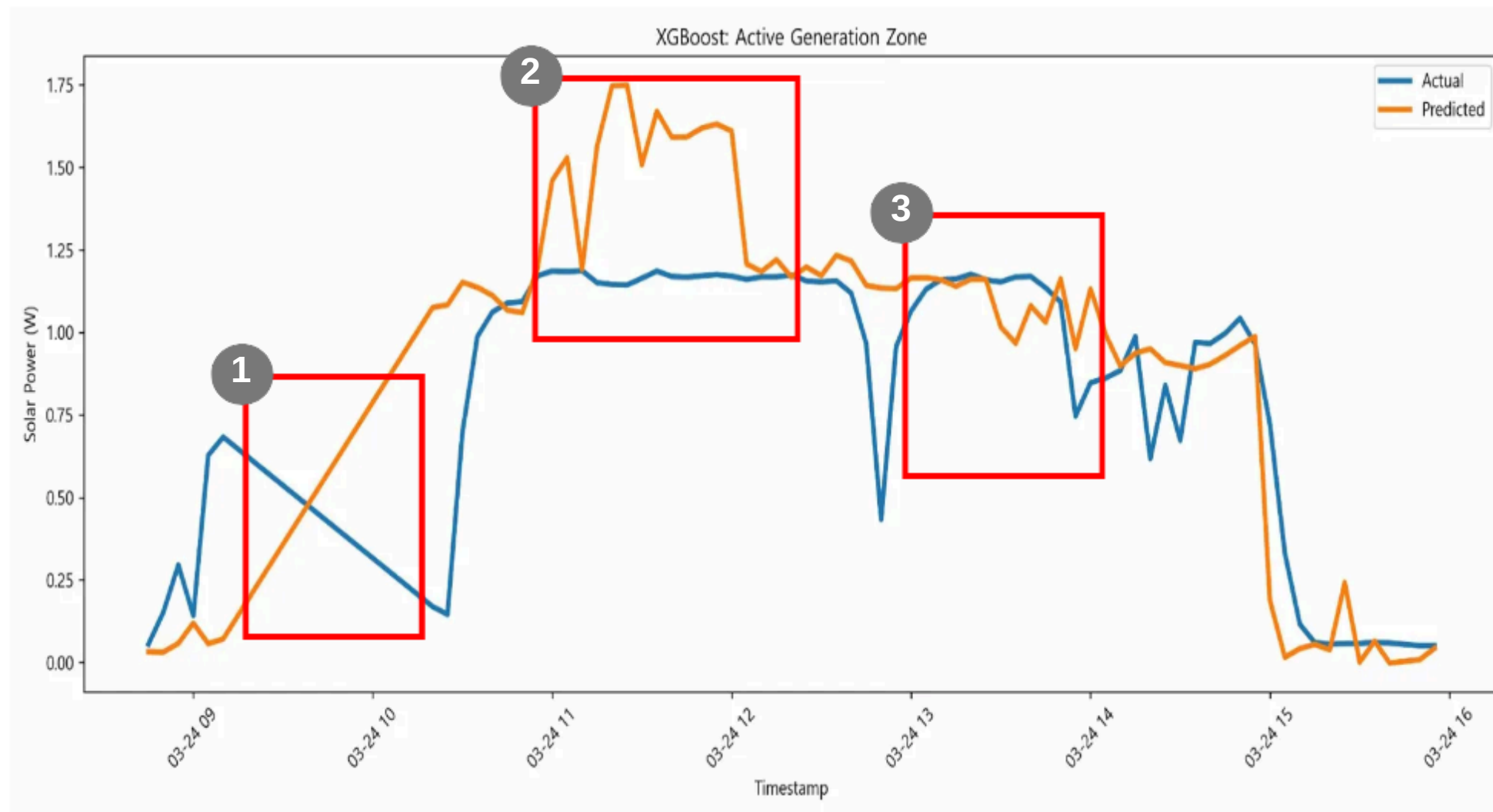
피처 엔지니어링 (총 52개)

- Lag: power_lag_1/2/3/6/12 (5~60분 전 발전량)
- Rolling: mean/std/max/min (3/6/12/24 window)
- 시간 인코딩: hour_sin/cos, minute_sin/cos
- 차분: power_diff_1/3/6 (발전량 변화율)

XGBoost 결과



Active Generation Zone, Feature Importance



1위: hour_cos (33%) → 시간대가 핵심 변수
2위: light_mean_3 (22%) → 최근 15분 평균 조도
3위: power_max_12 (13%) → 최근 1시간 최대 발전량
상위 3개가 전체 중요도의 70% 차지

1 발전 시작(09시): 예측이 실제보다 먼저 상승 → 발전 시작 시점 선행 감지 (hour_cos, light_mean_3)

2 11~12시: 예측이 실제보다 높은 과예측 경향

3 12~14시: 두 선 가장 근접 수렴 → 안정 발전 구간에서 높은 정확도

GRU vs XGBoost 비교



MAE (Mean Absolute Error, 평균 절대 오차):

예측이 평균적으로 얼마나 틀렸는지를 나타내는 숫자

	MAE	과거 처리	현재 성능
GRU	0.129W	모델 내부 자동	피크 구간 과예측 심함
XGBoost	0.105W	Lag/Rolling 직접 생성	상대적으로 안정적

결론: 현재 7일치 데이터 규모에서는 XGBoost가 더 안정적

장기간 다양한 날씨 조건 데이터가 쌓이면 시계열 패턴을 기억하는 GRU가 유리할 것으로 기대

04

web

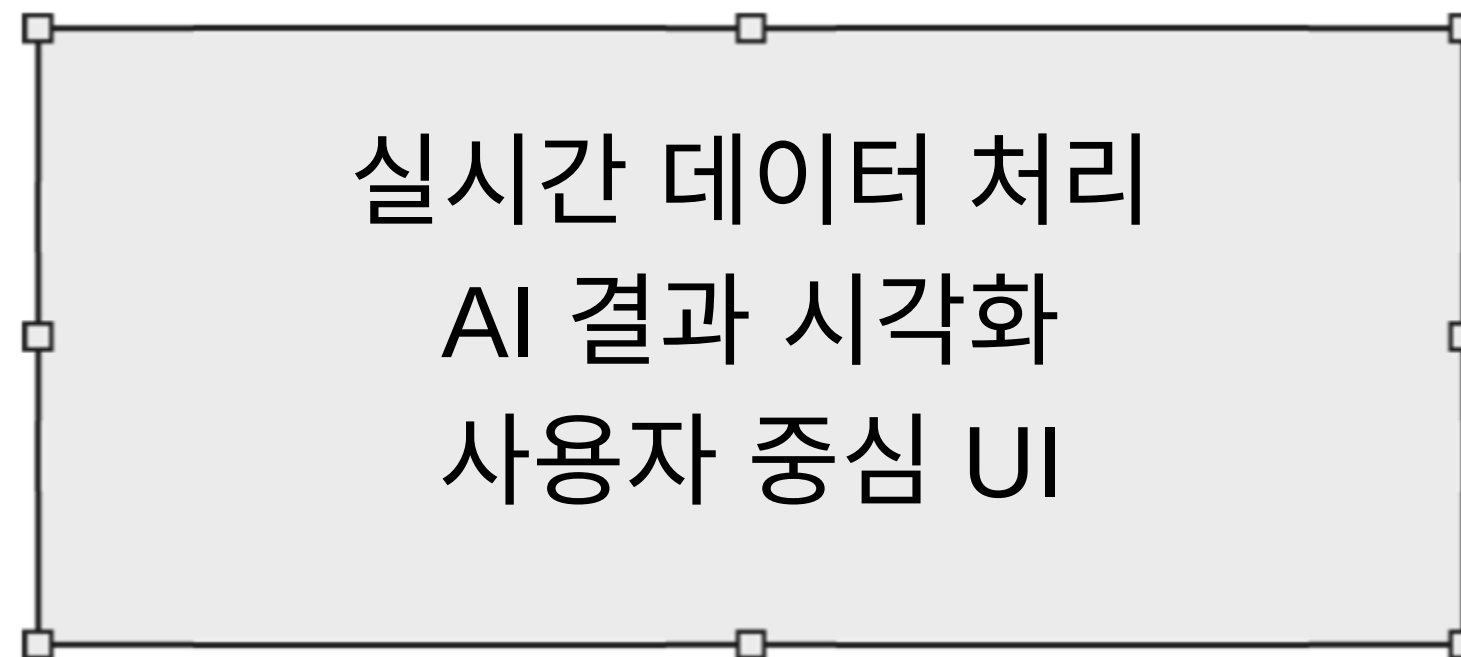
-
- 대시보드 설계
 - 데이터 흐름
 - 실시간 시연



웹 대시보드 설계



실시간 센서 데이터와 AI 예측 결과를
사용자가 직관적으로 확인할 수 있도록 설
계



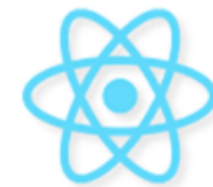
데이터 흐름



Arduino



Flask



REACT

Dashboard

Arduino로 받은 센서 데이터는 Flask 서버를 통해 전달되며,
웹에서 API를 통해 실시간으로 시각화 됨.

API 구조



api/data

센서 데이터 수신



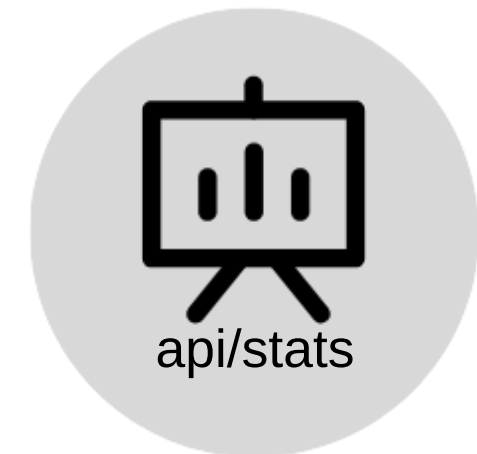
api/latest

현재 상태 조회



api/history

과제 데이터 세
고



api/stats

누적 통계 제공



```
"GET /api/history HTTP/1.1" 200 -  
"GET /api/stats HTTP/1.1" 200 -  
"GET /api/latest HTTP/1.1" 200 -  
"GET /api/health HTTP/1.1" 200 -
```

웹은 **REST API 구조**로 데이터를 받아 처리하며,
기능별로 분리된 **엔드포인트**를 통해 효율적으로 관리됩니다.

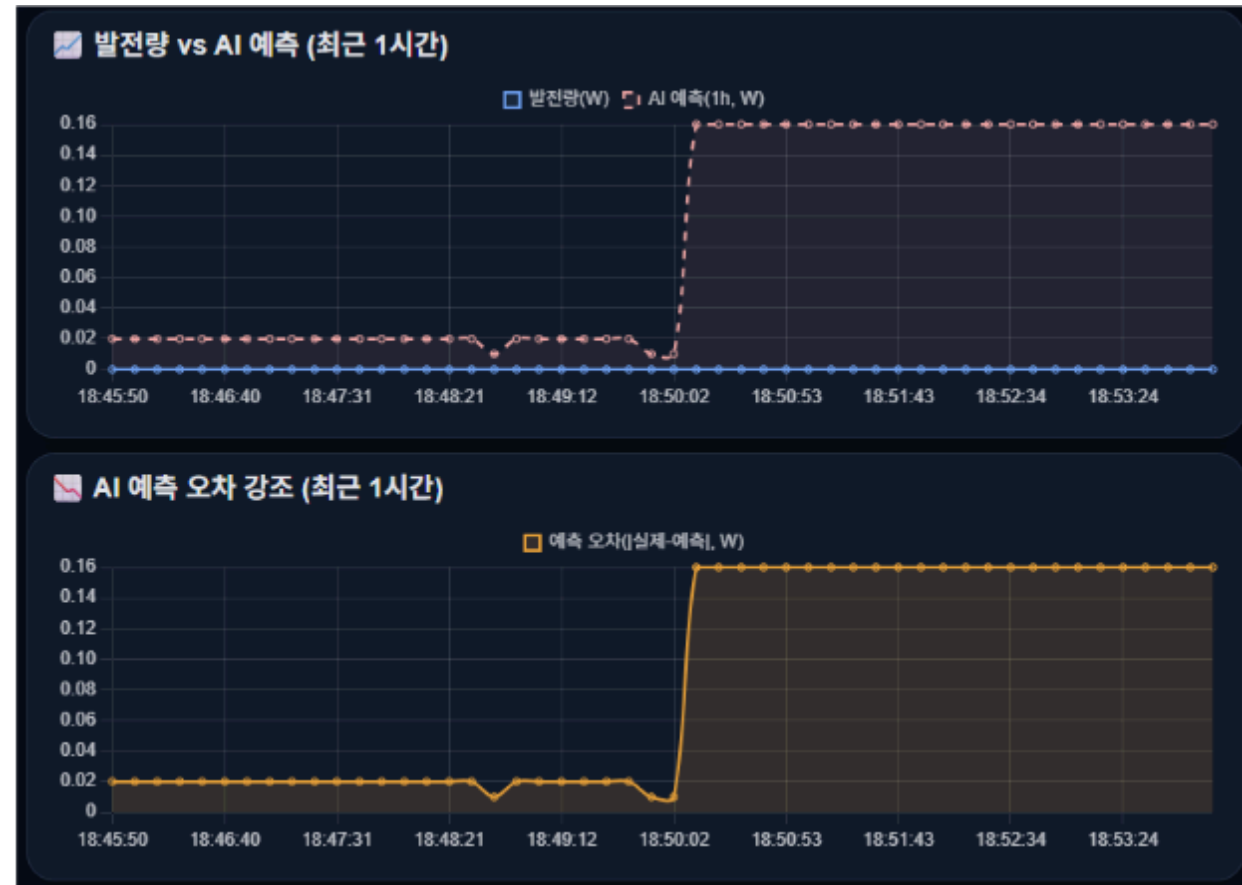
대시보드 UI



- 실시간 모니터링
 - 센서 데이터를 10초 주기로 갱신하여
현재 상태를 즉시 확인 가능
- AI 예측 제공
 - 1시간 후 발전량을 예측하여
에너지 사용 계획 수립 가능
- 상태 시각화
색상 기반으로 시스템 상태를
표현하여 직관적인 판단 가능

데이터를 분석하고, 즉시 행동할 수 있도록 지원

AI 신뢰도



발전량 vs AI 예측 그래프



오차, AI 신뢰도 카드

예측값과 실제값의 차이를 기반으로
AI 예측의 신뢰도를 계산합니다.

차이가 작을수록 신뢰도는 높고,
차이가 클수록 불확실성이 증가합니다.

이를 통해 사용자는 예측 결과를 판단 기준으로 활용할 수 있습니다.

UX 설계



정상



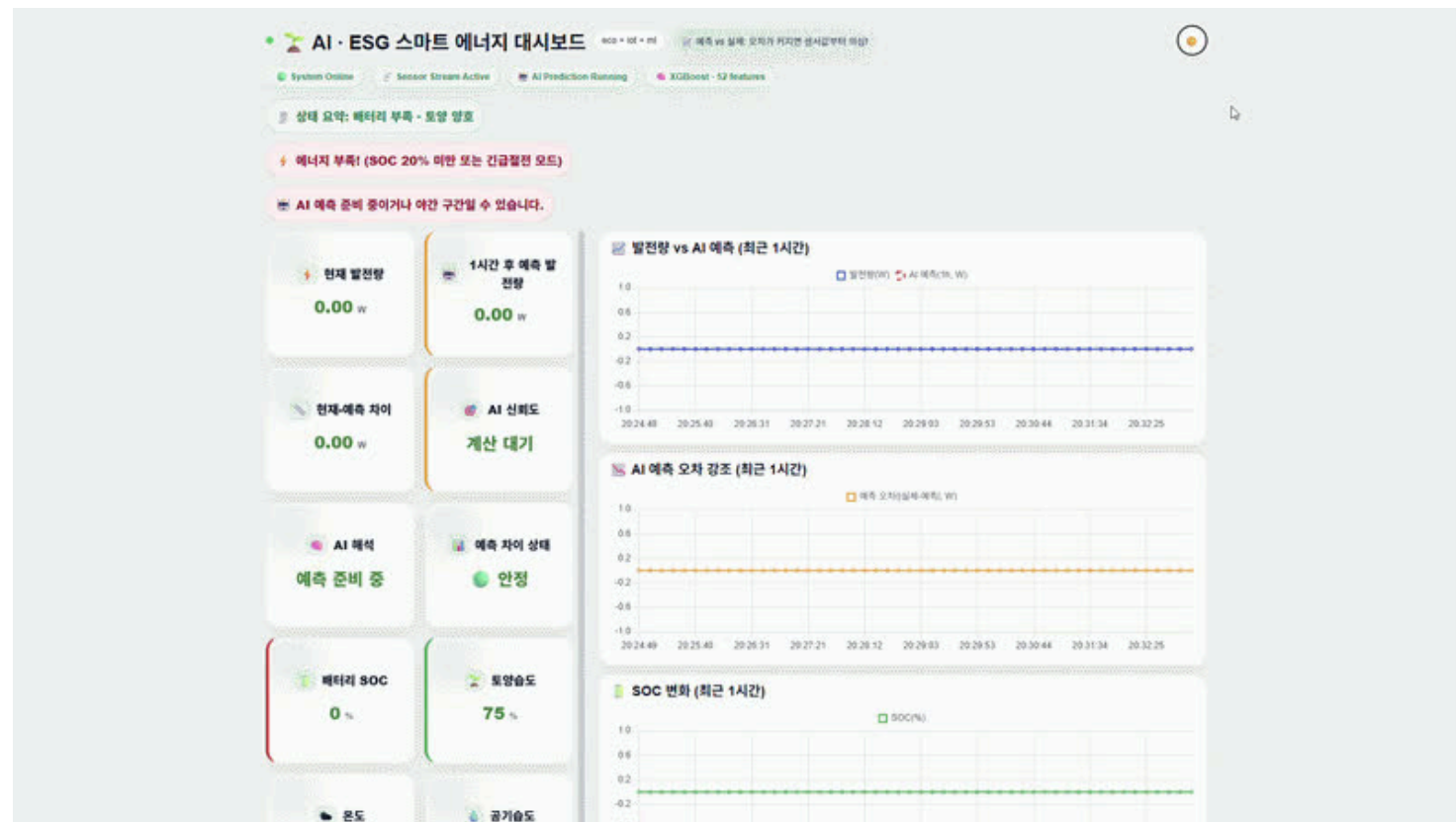
경고



위험



색상 기반으로 시스템 상태를
직관적으로 표현하여 사용자가
빠르게 상황을 인지할 수 있도록 설계

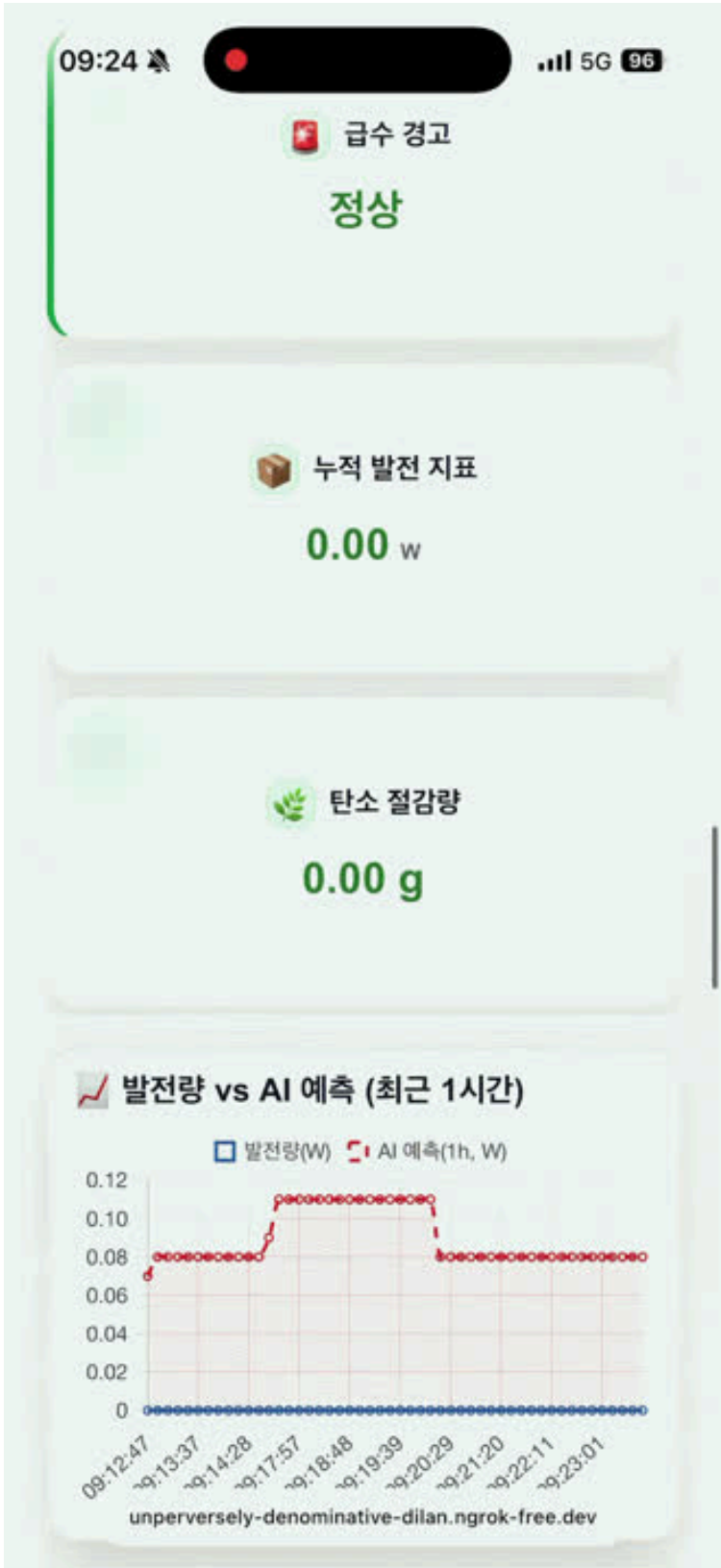


다크모드를 통해 사용자 환경에 맞는 UI 제
공

실시간 시연



Live Energy Monitoring



05

문제 정의 및 결론

-
- 문제 정의 & 한계 인식
 - 문제 해결 전략
 - 결론



문제 정의 & 한계 인식



시스템 한계

- 풍력발전 미적용
- ESP-01 모듈, 와이파이 연결 문제로 인한 단일 보드 구성

데이터 및 AI 한계

- 데이터 수 부족(7일)으로 GRU 성능 제한
- 정확한 시계열 학습에 한계 존재

에너지 한계

- 태양광 발전량 제한(출력 부족)으로 배터리 SOC 충전 효율 부족(충전 불안정)

문제 해결 전략



제한된 환경에서 최적의 선택을 하기 위한 전략적 의사 결정

데이터

데이터 부족으로 시계열 모델(GRU)
성능 한계 확인

XGBoost로 전략적 전환

에너지

태양광, 풍력 전력 공급의 불안정
태양광 패널 병렬 연결(소비 최적
화)

풍력발전 블레이드 소재 변경

WiFi

WiFi 모듈의 발열 및 전력 불안정

안정적인 통신 구조 필요
라즈베리파이 / ESP32 구조 추가

AI

급변하는 발전량 패턴 학습 한계
단일 시계열 구조의 한계

Lag / Rolling / Diff 기반
Feature Engineering 적용

AI 기반 제어로의 전환

현재는 안정성을 위한 룰 기반,
미래는 AI 중심의 예측 기반 제어

현재

SOC 기준 룰 기반 제어

- 현재 상태에 반응하는 방식
- 단순 조건 기반 제어
- 사후 대응 중심

미래

AI 예측 기반 제어

- 발전량 예측 기반 의사결정
- 미래 상황을 고려한 제어
- 선제적 에너지 최적화

에너지 부족 상황을 사전에
대응
시스템 안정성 확보

EX)



확장 가능한 AI 에너지 관리 시스템



시스템 한계

- 계절별 데이터 수집(여름 / 겨울)
- 공공 태양광 및 기상 데이터 활용
- 장기 데이터 확보를 통한 학습 안정성 향상

데이터 및 AI 한계

- 라즈베리파이 기반 독립 서버 구축
- ESP-32 통합 제어 구조 적용
- 외부 전원 없이 자율 운영 시스템 구현

에너지 한계

- 데이터 증가 시 GRU / LSTM 재도입
- XGBoost + 시계열 모델
- AI 신뢰도 기반 자동 제어 적용

데이터가 축적될수록 성능이 향상되는,
지속적으로 학습하는 에너지 관리 시스템



AI 기반 재생에너지 스마트팜

예 측 · 최 적 화 · 자 동 화

지속 가능한 에너지 운영을 위한 시스템



Thank you

들어주셔서 감사합니다.